Mapa doboru podejścia: synchronia, wątki, async i procesy w Pythonie

Cel dokumentu

Celem dokumentu jest pomoc w wyborze właściwego podejścia do organizacji pracy programu w Pythonie. Program może działać synchronicznie, wielowątkowo, asynchronicznie lub wieloprocesowo. Każde z tych podejść rozwiązuje inny typ problemu.

Najważniejsze pytanie

Zanim wybierzemy technikę, trzeba odpowiedzieć na pytanie: gdzie program traci czas? Jeśli głównie czeka na sieć lub pliki (I/O), potrzebujemy współbieżności. Jeśli głównie wykonuje obliczenia, potrzebujemy wykorzystać wiele rdzeni CPU.

I/O-bound – program czeka

Typowe przykłady: pobieranie stron WWW, komunikacja z API, wysyłanie maili SMTP, pobieranie plików FTP/SFTP, zapisy do bazy danych. W takich zadaniach CPU często jest bezczynne i czeka na odpowiedź z zewnątrz.

CPU-bound – program liczy

Typowe przykłady: analiza obrazów, obliczenia statystyczne, przetwarzanie NLP, kompresja danych, obliczanie hashy. W tym przypadku procesor jest głównym ograniczeniem wydajności.

Synchronia

Najprostszy model programowania. Instrukcje wykonują się jedna po drugiej. Stosuj go, gdy zadanie jest małe, liczba operacji niewielka, a prostota kodu jest ważniejsza niż maksymalna wydajność.

Wielowątkowość

Wątki są dobrym rozwiązaniem dla blokujących operacji I/O. Przykłady zastosowań: pobieranie wielu stron HTTP, transfer plików SFTP/SCP, wysyłanie wielu maili SMTP. Wątki pozwalają wykonywać wiele operacji sieciowych równocześnie.

Asynchroniczność

Async pozwala obsługiwać wiele operacji oczekujących w jednym wątku przy użyciu event loop. Najlepiej sprawdza się w aplikacjach sieciowych o dużej skali: crawlerach WWW, serwerach API, systemach obsługujących setki połączeń.

Wieloprocesowość

Gdy zadanie jest CPU-bound, najlepszym rozwiązaniem jest multiprocessing. Każdy proces działa na osobnym rdzeniu procesora, dzięki czemu można realnie przyspieszyć ciężkie obliczenia.

Mapa decyzyjna

Jeśli program głównie czeka → użyj async lub wątków.

Jeśli program głównie liczy → użyj procesów.

Jeśli zadanie jest małe → użyj kodu synchronicznego.

Typowe scenariusze

- Pobieranie wielu stron WWW – async
- Transfer wielu plików SFTP – wątki
- Analiza obrazów – procesy
- Crawler + analiza treści – async + procesy
- Wysyłanie wielu maili – wątki

Najważniejsza reguła

Sieć i operacje I/O łączymy najczęściej z async lub wątkami. Ciężkie obliczenia przenosimy do procesów.